

Customer Churn Prediction Using Decision Tree Classifier

Dr. Anilkumar V. Brahmane

Department of Computer Engineering

Sanjivani College of Engineering

Course: Machine Learning Assignment

Application Domain: Telecom Industry

Dataset: IBM Telco Customer Churn (Kaggle)

Objectives of this Presentation:

- Understand the business problem of customer churn
- Explore the IBM Telco dataset (7,043 rows × 21 columns)
- Build a Decision Tree Classifier in Python step-by-step
- Evaluate and interpret model performance metrics
- Visualize and trace the decision tree logic

Machine Learning Assignment | Telecom Industry Application

What is Customer Churn? — The Business Problem

Definition: Customer Churn means a customer **stops using a company's service** and moves to a competitor. In telecom, this is a critical business problem that directly impacts revenue and growth.

Why It Matters — The Numbers

5–7×

Acquisition Cost

More expensive to acquire a new customer than to retain an existing one

25%

Annual Churn Rate

Telecom companies lose 15–25% of customers annually — millions in lost revenue

Goal of This Assignment

- **Predict:** Which customer will churn (Yes/No) *before* it happens, so the company can take preventive action
- **Why Decision Tree?** Easy to understand, visualize, and explain — ideal for business stakeholders and students alike
- **Real-world use:** Telecom companies use such models to offer discounts, upgrade plans, or assign retention agents to at-risk customers

Problem Type

Binary Classification — Churn = Yes (1) or No (0)

Input

Customer demographics, contract details, service usage, billing data

Output

Prediction: Will this customer churn in the next period? Yes or No

Business Action

Proactive retention — offer loyalty bonus, discounted plan, or assign retention agent

📌 Bottom line: This is a **Binary Classification Problem** — Churn = Yes (1) or No (0). Predicting churn early saves revenue.

Dataset Overview — IBM Telco Customer Churn Dataset

Source: IBM Sample Dataset (available on Kaggle: *Telco Customer Churn*) | **Size:** 7,043 rows × 21 columns

Key Input Features (X — Independent Variables)

Numerical Features

- tenure:** Months with company (1–72)
- MonthlyCharges:** Amount charged/month (₹29.85–₹118.75)
- TotalCharges:** Total amount charged over tenure
- SeniorCitizen:** 0 = No, 1 = Yes

Categorical Features

- Contract:** Month-to-month, One year, Two year
- InternetService:** DSL, Fiber optic, None
- PaymentMethod:** Electronic check, Mailed check, Bank transfer, Credit card
- TechSupport / OnlineSecurity:** Yes / No
- Partner / Dependents / PaperlessBilling:** Yes / No

Sample Dataset Rows

customer ID	tenure	MonthlyCharges	Contract	Churn
7590-VHVEG	1	29.85	Month-to-month	No
3668-QPYBK	2	53.85	Month-to-month	Yes
5575-GNVDE	34	56.95	One year	No
7795-CFOCW	45	42.30	One year	No
9237-HQITU	2	70.70	Month-to-month	Yes

Target Variable (Y — Output)

Churn = "Yes" → 1

Customer LEFT the company

Churn = "No" → 0

Customer STAYED with the company



Class Distribution: ~73% No Churn (5,174), ~27% Churn (1,869) — slightly imbalanced dataset. This affects choice of evaluation metric.

Machine Learning Workflow — Step-by-Step Pipeline

The complete pipeline from raw data to final prediction follows these 7 stages. We train on 80% of data and test on the remaining 20% — the model **never sees test data during training**.

01	02	03
Data Collection	Exploratory Data Analysis (EDA)	Data Preprocessing
Load IBM Telco Churn CSV using Pandas — 7,043 customer records with 21 features	Understand data shape, check for missing values, visualize churn distribution using bar charts and heatmaps	Handle missing values (TotalCharges has blanks), encode categoricals (Label/One-Hot Encoding), drop irrelevant columns (customerID)
04	05	06
Feature Selection	Train-Test Split	Model Training
Select input features (X) and target variable (y = Churn). All 20 remaining columns used as features after dropping customerID	80% Training (5,634 rows) and 20% Testing (1,409 rows) using <code>train_test_split()</code> with <code>random_state=42</code>	Create <code>DecisionTreeClassifier()</code> object and fit on training data — the tree learns decision rules automatically from features
07		
Model Evaluation		
Predict on test data → Calculate Accuracy, Confusion Matrix, F1-Score → Visualize the Decision Tree structure		

  **Key Principle:** Training data teaches the model; test data measures how well the model generalizes to unseen customers.

Decision Tree Algorithm — How It Works

A **Decision Tree** is a **tree-shaped flowchart** where each internal node tests a feature, each branch represents an outcome, and each leaf node gives the final prediction (Churn: Yes/No). The algorithm used is **CART — Classification and Regression Trees**.

Core Concepts of the CART Algorithm

Splitting Criterion — Gini Impurity

Measures how "impure" a node is. **Gini = 0** means perfectly pure (all same class). The algorithm picks the feature + threshold that gives the **lowest Gini impurity** after the split.

Root Node

The very first split — the most important feature (e.g., Contract type) that best separates churners from non-churners across the entire training dataset.

Internal Nodes

Each subsequent split on a feature (e.g., `tenure < 12?` `MonthlyCharges > 65?`). The tree asks yes/no questions at each level.

Leaf Nodes

Final decision — "Churn = Yes" or "Churn = No" based on majority class in that group. No further splits occur here.

max_depth Parameter

Controls how deep the tree grows. Deeper = more complex = risk of overfitting. **Typical value: 3–5** for interpretability and generalization.

Entropy (Alternative)

Another splitting criterion using information gain. Both Gini and Entropy give similar results in practice — Gini is computationally faster.

Simple Decision Rules Learned by Tree

● High Churn Risk

IF Contract = "Month-to-month"
AND tenure < 12
AND MonthlyCharges > 65
→ **Churn = YES**

● Low Churn Risk

IF Contract = "Two year"
AND tenure > 24
→ **Churn = NO**

Why Decision Trees Are Preferred for This Problem

- **Interpretable:** Rules can be read and explained in plain English
- **No scaling needed:** Unlike SVM or KNN, no feature normalization required
- **Handles mixed data:** Works with both numerical (tenure) and categorical (Contract) features
- **Visual output:** Tree diagram can be shown to business managers directly

Python Code — Step 1 to 3: Import, Load & Explore Data

Step 1: Import All Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score,
                             confusion_matrix, f1_score,
                             classification_report)
from sklearn import tree
```

pandas as pd

Load and manipulate dataset as a DataFrame (table format) — the primary data handling library

matplotlib / seaborn

Plot graphs, visualize the decision tree diagram, and create heatmaps for confusion matrix

train_test_split

Splits data into training and testing sets automatically — essential for unbiased evaluation

numpy as np

Numerical operations on arrays — required by sklearn internally

DecisionTreeClassifier

The main ML model class from scikit-learn — this is what we train and use for prediction

metrics functions

accuracy_score, confusion_matrix, f1_score, classification_report — all evaluation tools

Step 2: Load Dataset

```
df = pd.read_csv('Telco-Customer-Churn.csv')
df.head()
df.shape
df.info()
df['Churn'].value_counts()
```

pd.read_csv(...)

Reads the CSV file into a DataFrame called df — 7,043 rows, 21 columns loaded into memory

df.shape

Returns (7043, 21) — confirms 7,043 rows and 21 columns are present

df['Churn'].value_counts()

Shows: No = 5,174 | Yes = 1,869 — reveals class imbalance (73% vs 27%)

df.head()

Shows first 5 rows to verify data loaded correctly — quick sanity check on column names and values

df.info()

Shows column names, data types, and non-null counts — helps identify missing values and wrong data types

Python Code — Step 4 to 5: Preprocessing & Feature Engineering

Step 4: Data Cleaning & Encoding

```
df.drop('customerID', axis=1, inplace=True)

df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'], errors='coerce')

df.dropna(inplace=True)

df['Churn'] = df['Churn'].map(
    {'Yes': 1, 'No': 0})

df = pd.get_dummies(df, drop_first=True)
```

drop('customerID', ...)

Remove customerID — it is just an identifier, not useful for prediction. Keeping it would confuse the model

pd.to_numeric(..., errors='coerce')

TotalCharges is stored as text (string) — convert to number. Invalid entries (spaces) become NaN automatically

dropna(inplace=True)

Remove rows with missing values — only **11 rows** affected (those with blank TotalCharges)

map({'Yes': 1, 'No': 0})

Convert target variable: Yes→1, No→0. Machine learning models understand numbers, not text labels

pd.get_dummies(..., drop_first=True)

One-Hot Encoding: converts ALL categorical columns (Contract, PaymentMethod, etc.) into binary 0/1 columns. drop_first=True avoids multicollinearity

Step 5: Define Features, Target & Split Data

```
X = df.drop('Churn', axis=1)
y = df['Churn']

X_train, X_test, y_train, y_test = \
    train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )
```

X = df.drop('Churn', ...)

X = all columns except Churn — these are the input features (independent variables) fed into the model

y = df['Churn']

y = only the Churn column — this is what we want to predict (dependent variable / target)

test_size=0.2

20% of data (≈1,407 rows) reserved for testing. 80% (≈5,625 rows) used for training the model

random_state=42

Ensures the same split every time you run the code — reproducibility is essential for experiments

X_train

5,625 rows — model learns from this

X_test

1,407 rows — model never sees this during training

Python Code — Step 6: Train the Decision Tree Model

Step 6: Create, Train, and Predict

```
dt_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)

y_pred = dt_model.predict(X_test)

print(dt_model.feature_importances_)
```

`criterion='gini'`

Use Gini Impurity to decide the best split at each node — measures class mixing; lower is better

`random_state=42`

Ensures reproducible results every time you run the code — essential for assignment submission

`dt_model.predict(X_test)`

Prediction step: Apply learned rules to test data. For each of 1,407 test customers, model outputs 0 (No Churn) or 1 (Churn)

`max_depth=5`

Limit tree to 5 levels deep — prevents overfitting and keeps tree readable/interpretable for humans

`dt_model.fit(X_train, y_train)`

Training step: The model learns decision rules from training data — finds best feature splits to separate churners from non-churners

`feature_importances_`

Shows which features the tree used most — typically Contract, tenure, MonthlyCharges are top contributors

Visualize the Decision Tree

```
plt.figure(figsize=(20, 10))

tree.plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=['No Churn', 'Churn'],
    filled=True,
    fontsize=10
)

plt.savefig('decision_tree.png', dpi=150)
plt.show()
```

`plt.figure(figsize=(20,10))`

Create a large figure — the decision tree diagram is wide and needs ample space to display all nodes clearly

`feature_names=X.columns`

Labels each node with the actual feature name (e.g., "tenure ≤ 14.5") instead of a number index

`plt.savefig(..., dpi=150)`

Save the tree image at high resolution for inclusion in your assignment report or presentation

`tree.plot_tree(...)`

Draws the full trained tree with colored nodes. `filled=True` colors nodes by class: blue/teal = No Churn, orange = Churn

`class_names=['No Churn','Churn']`

Labels leaf nodes with readable class names instead of 0 and 1 — makes tree output self-explanatory

TREE VISUALIZATION

Actual Decision Tree — How to Read Each Node Box

After running

Decision Tree — Tracing a Customer's Path (Example)

Let's trace how the tree predicts churn for a real customer example, step by step through the tree nodes:

Contract
Month-to-month

Tenure
3 months

MonthlyCharges
₹85

TechSupport
No

InternetService
Fiber optic

Node	Condition	Customer Value	Direction	Prediction
Root	Contract_Two year ≤ 0.5	0 (Month-to-month)	LEFT ✓	—
Node 2	tenure ≤ 14.5	3 months	LEFT ✓	—
Node 3	MonthlyCharges ≤ 64.35	₹85	RIGHT ✓	—
Node 4	Fiber optic ≤ 0.5	1 (Yes)	RIGHT ✓	—
Leaf	—	—	STOP	CHURN ⚠

Reading the Leaf Node

- **Gini = 0.38** — moderately pure node (churners dominate)
- **Samples = 410** — 410 training customers reached this leaf
- **Value = [155, 255]** — 155 stayed, 255 churned
- **Class = Churn** — majority class → prediction is CHURN

Business Insight from This Path

Risk Profile Identified

Month-to-month + short tenure (3 months) + high charges (₹85) + Fiber optic = **highest churn risk combination**

Recommended Action

Offer this customer a discounted annual plan or loyalty bonus **immediately** — before they leave

FEATURE IMPORTANCE

Decision Tree — Feature Importance & Tree Structure Summary

After training, we extract which features the tree relied on most using

Python Code — Step 7: Model Evaluation Metrics

Accuracy Score & F1-Score

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")

f1 = f1_score(y_test, y_pred)
print(f"F1-Score: {f1:.4f}")

print(classification_report(
    y_test, y_pred))
```

~80%

Accuracy

~4 out of 5 customers correctly classified (78–82% typical range)

~0.60

F1-Score (Churn)

Balances Precision & Recall for the minority Churn class (0.55–0.65 typical)

❏ Why F1? Dataset is imbalanced (73% No, 27% Yes) – accuracy alone is misleading. A model that always predicts "No Churn" gets 73% accuracy but detects zero churners!

Confusion Matrix Code

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d',
    cmap='Blues',
    xticklabels=['No Churn','Churn'],
    yticklabels=['No Churn','Churn'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Confusion Matrix — Interpretation

	Predicted: No Churn	Predicted: Churn
Actual: No Churn	TN = ~950 ✓ (Correct)	FP = ~80 ✗ (False Alarm)
Actual: Churn	FN = ~220 ✗ (Missed!)	TP = ~157 ✓ (Correct)

TN — True Negative (~950)

Correctly predicted "No Churn" – customer actually stayed. ✓ Good outcome.

TP — True Positive (~157)

Correctly predicted "Churn" – customer actually left. ✓ Model caught the churner.

FP — False Positive (~80)

Predicted Churn but customer stayed (Type I Error). Minor cost – we offer retention to someone who didn't need it.

FN — False Negative (~220)

Predicted No Churn but customer left (Type II Error). **Most costly!** We missed an at-risk customer entirely.

⚠️ FN (False Negatives) are the most dangerous error in churn prediction – every missed churner represents lost revenue. Minimize FN by tuning threshold or using `class_weight='balanced'`.

Results, Interpretation & Key Takeaways

Model Performance Summary

~81%

Accuracy

Correctly classifies ~4 out of 5 customers (79–82% range)

~0.62

F1-Score

Churn class F1 — reasonable for imbalanced dataset (0.58–0.65)

Top Predictive Features Learned

Contract Type

Month-to-month = highest risk.
Two-year = lowest risk.

Tenure

Short tenure (<12 months) =
3× higher churn likelihood

MonthlyCharges

High charges (>₹65) + short
tenure = very high risk
combination

TechSupport

No support subscription =
higher churn when issues arise

How to Improve the Model

→ Tune max_depth

Try depth 3, 4, 5, 6 — use cross-validation to find the optimal value that balances accuracy and overfitting

→ Handle Class Imbalance

Use `class_weight='balanced'` in `DecisionTreeClassifier` to give more importance to minority class (Churn=Yes)

→ Try Random Forest

Ensemble of 100+ decision trees — typically gives **85–88% accuracy** on this dataset with better generalization

→ Feature Engineering

Create new features like "charges per month of tenure" to improve predictive signal

Assignment Deliverables Checklist

- ✓ Decision Tree model built and trained on IBM Telco dataset (7,043 rows)
- ✓ Accuracy, Confusion Matrix, and F1-Score computed and interpreted
- ✓ Decision Tree visualized using `tree.plot_tree()` with `filled=True`
- ✓ All Python steps explained with meaning and purpose of each line
- ✓ Feature importance chart generated and top predictors identified
- ✓ Customer decision path traced step-by-step through tree nodes
- ✓ Business insights derived from model rules for real-world action

📌 Conclusion

Decision Trees are **powerful, interpretable, and business-friendly**. The rules they learn — e.g., "Month-to-month + high charges + short tenure = churn risk" — can be **directly acted upon** by telecom companies without needing a data scientist to explain every prediction.

Unlike black-box models, every Decision Tree prediction can be traced to a clear IF-THEN rule — making it the ideal starting point for churn prediction in industry.